Which Layer Catches What? An Adversary-Stratified Tamper Benchmark for AI-Agent Negotiation Records[1]

Bamio

Independent researcher

With

Apart Research

Abstract

AI agents increasingly negotiate for people, and the record of what they agreed is often the only evidence others can check. Record verifiers usually show one green verdict, but what that verdict licenses a reader to believe is rarely measured. We built an adversary-stratified mutation benchmark for Blackbox, a preexisting system in which two LLM agents negotiate through a deterministic referee, with a hash-chained log, committed private limits, signed offers, a replay verifier and a Solana memo anchor. We generated 200 honest receipts, applied 26 targeted attack operators in five adversary classes plus pre-anchor, fuzz and benign controls (6,756 mutants), and scored seven verifier configurations. No configuration flagged any of 600 benign controls. The shipped verifier flagged 42.6% of harmful mutants offline and 70.1% with its on-chain anchor check. Three gaps explain most misses. First, the receipt tells the verifier where its anchor is, so deleting the anchor or pointing it at an attacker-written memo passes. Second, the summary fields shown to users are not bound to the log. Third, removing newer evidence downgrades checks to "pending". Pinning the anchor wallet and looking anchors up by deal ID, binding summary fields, and refusing silent downgrades raised detection to 98.1%. A live run on a real Solana-mainnet deal reproduced the gaps. Forgery by the anchoring operator before anchoring remains undetectable. Tamper evidence for agent records depends less on in-record cryptography than on where verifiers look for external evidence.

1 Research conducted at the AI x Epistemics Research Sprint, November 2026

1. Introduction

AI agents are starting to negotiate, buy and sell on behalf of people. When two agents strike a deal, the parties, their owners and any later arbiter need to know what was agreed, how and whether the rules were followed. The record of the negotiation becomes evidence. Forethought argue that auditability is a design requirement for epistemic tools, because early tools accumulate trust whether or not they deserve it [9]. A record verifier is such a tool. It turns a complex log into one verdict that people and other agents will rely on.

A verdict like "Record holds up" is itself a claim, and its meaning depends on the adversary. A hash chain catches someone who edits a line and forgets to recompute hashes. It does not catch someone who recomputes them. Signatures, commitments, deterministic replay and external anchors each close some gaps and leave others. In practice verifiers also have legacy paths, optional fields and summaries for display, and each of these can quietly weaken the guarantee. Few deployed agent systems report which adversaries their green badge actually excludes.

We study this question on **Blackbox**, a preexisting agent-negotiation system (Section 3.1). In Blackbox, two LLM agents haggle through a deterministic referee, every event is hash-chained, private limits are committed before the first offer, offers are signed, a public verifier replays the referee, and the final hash is anchored on Solana mainnet as an SPL Memo. During the sprint we built an adversary-stratified mutation benchmark around Blackbox's verifier and measured which evidence layer catches which adversary.

Our main contributions are:

1. **BlackboxTamperBench.** We release a reproducible benchmark with 26 targeted attack operators in five adversary classes, a pre-anchor forgery operator, two random-fuzz operators and three benign controls. Applied to 200 honest receipts, it yields 6,756 mutants. It runs in under a minute with no API calls and no money spent.
2. **A layer ablation of a deployed verifier.** Detection by the shipped verifier is 42.6% offline and 70.1% with its on-chain anchor, and false positives are 0. We trace the misses to three concrete gaps: a receipt-directed anchor check, unbound summary fields and silent downgrades. We also identify claims that no configuration can verify.
3. **Hardenings with measured gains, validated live.** Three small changes raise detection to 98.1% with 0 of 600 benign controls flagged. Their cost is that 3 of 4 real legacy receipts become "unverifiable". Replaying the attacks against a real deal on Solana mainnet reproduces the gaps.

2. Related Work

Tamper-evident logging. Secure audit logs [1] and Merkle-tree history structures [2] make logs tamper-evident against an untrusted logger, given auditors who hold earlier commitments. Crosby and Wallach stress that without regular auditing, a logger can drop or rewrite events undetected [2]. Our results are an empirical instance of that point: in-record checks catch none of the full forgeries we tested, and only an anchor the verifier looks up for itself catches all of them.

Timestamping and transparency logs. Linking a document hash to a widely witnessed record dates back to Haber and Stornetta [3]. Certificate Transparency [4] and Sigstore's Rekor [5] apply the same idea at scale. Blackbox's Solana memo is such a witness. We show that the guarantee depends on how the verifier locates the witness. If the record under test says where its own anchor is, the anchor can be removed or replaced.

Provenance standards for AI. C2PA [6] binds provenance claims to media through signed manifests. The OpenTelemetry GenAI semantic conventions [7] standardise traces of what LLM agents did, but they do not define integrity protection. The sprint's Track 2 asks which minimal tamper-evident layer is missing from such standards. Our layer ablation answers that question for one concrete agent-record format.

Multi-agent risk and injection. Hammond et al. list commitment problems and multi-agent security as key risk factors for interacting agents [8]. Indirect prompt injection [10] motivates Blackbox's schema-only referee. In our benchmark, records of blocked injections can be deleted, or invented to frame the other party, and in-record checks catch neither.

Method. We borrow mutation analysis from software testing [11, 12]. Instead of mutating programs to score a test suite, we mutate records to score a verifier. Our "downgrade" class follows downgrade attacks on security protocols [13], where an attacker strips newer evidence so that a weaker legacy check applies. As far as we know, no prior work reports adversary-stratified verdict soundness for a deployed AI-agent negotiation verifier.

3. Methods

3.1 System under test (preexisting work)

A Blackbox receipt contains a hash-chained log (`deal.open`, `agent.identity`, `mandate.open`, `commit`, sealed `prompt`/`response` hashes, `offer.accepted` / `offer.blocked` / `offer.rejected`, `round.closed`, `deal.closed` or `deal.no_deal`, `reveal`, and `settle.*`). It also contains summary fields shown on the receipt card (`outcome`, `terms`, `reveals`, `budget`, `identity`) and an `anchor` field. The **shipped verifier** (C3 offline, C4 with `--onchain`) recomputes the hash chain and final hash, compares the referee code hash with the published one, checks the round policy, replays the referee and audits each entry, checks that limits were committed before any offer and that the reveals open those commitments, checks that the price lies within both limits, checks the EIP-191 signature on every offer, and checks the spending-limit commitment and session caps. With `--onchain` it also re-reads the payment and the anchor memo from Solana. A check can pass, fail or be pending. Like the shipped CLI and UI, we count a record as **"holds up"** when no check fails. All of this code existed before the sprint and is used unmodified.

3.2 Honest corpus

We ran the unmodified `negotiate()` pipeline 200 times, with seed 20261113. Scripted, randomised agents stood in for the LLM. The model is not under test, and a scripted agent avoids API cost. Scenarios drew a reference price of 0.2–20.2 USDC and a quantity of 1–500, with overlapping limits in about 80% of cases, 1–3 rounds requested by each side (the referee uses the lower), random negotiating styles, and simulated attack mode (an injected message plus a backwards bid) with probability 0.3. The result was 112 closed deals and 88 no-deals. 22 receipts recorded refusals. Each honest final hash was written to a **simulated anchor registry** that models Blackbox's SPL memos (`blackbox:<dealId>:<finalHash>`, signed by the official anchor wallet). We repeated the experiment on a second corpus (100 receipts, seed 424242). We also ran it on the 4 real receipts produced by Blackbox's LLM agents (gpt-5.4-mini via SERV Reasoning), one of which settled 0.86 USDC on Solana mainnet.

3.3 Adversary classes and mutation operators

Table 1. Adversary classes and operators. A1–A5 are in scope for every configuration. P is out of scope by construction and marks the trust boundary. Z is unrestricted and not all of its changes are harmful.

Class	Capability	Operators (n mutants on the 200-receipt corpus)
A0 benign	content unchanged	unchanged (200); re-serialised with shuffled keys (200); anchor field removed, content intact (200)
A1 naive edit	edits JSON, no rehash	offer price, closing price, revealed limit, delete entry, swap entries, timestamp (1,112)
A2 chain rewrite	edits, then recomputes every hash and the receipt's own final and anchor hashes	offer price, closing price + summary, revealed limit, referee hash, hide refusals, frame an injection, backdate all, truncate tail (1,332)
A3 full forgery	also mints fresh agent keys, re-signs, recomputes commitments and summary	all prices and limits $\times 1.10$; $\times 1.10$ + strip anchor; $\times 1.10$ + self-written memo; rewrite both limits; limits + self-written memo (1,000)
A4 downgrade	removes newer evidence, rehashes	strip agent identities and signatures; strip reveals (400)
A5 summary edit	edits the fields the receipt card shows	outcome price, outcome status, terms, budget limit, reveals (912)
P pre-anchor	anchoring operator forges before anchoring	consistent $\times 1.10$ forgery, anchored by the official wallet (200)
Z fuzz	random leaf change in a random entry	without rehash (600); with rehash (600)

3.4 Verifier configurations

We test cumulative layers. **C1** checks only the hash chain and final hash. **C2** adds the referee replay and the per-entry rule audit. **C3** is the shipped verifier offline. **C4** is the shipped verifier with the anchor check **as shipped**: it follows the transaction and writer named in the receipt, and a missing anchor counts as pending. Three hardenings were written during the sprint. **Summary binding** requires every displayed summary field to equal its value derived from the log. The **strict profile** requires identities, the spending mandate, reveals and an outcome to be present, and counts a pending signature or reveal check as a failure. The **pinned anchor lookup** ignores the receipt's anchor field, takes the earliest memo for this deal ID from the published anchor wallet, and requires it to match. **C3h** is C3 with summary binding and the strict profile. **C5** is C4 with the same two additions, and **C6** is C5 plus the pinned lookup.

3.5 Metrics and live validation

The detection rate is the share of harmful mutants that do not receive "holds up". The false-positive rate is the share of benign controls that are flagged. We report Wilson 95% intervals, and we call an operator *fully caught* when at least 99% of its mutants are flagged. For live validation, we ran 13 cases on the real deal `deal_74b9a05fe8f4` with the shipped verifier's live Solana reads (read-only RPC). We also built a pinned lookup over the official anchor wallet's real transaction history. For the self-anchoring case we did not write an attacker memo on mainnet. That one transaction was served from a local stub, and every other call went to Solana mainnet.

4. Results

Table 2. Verdict soundness by configuration, on 200 honest receipts and 6,756 mutants. Pooled rates depend on the operator mix, so the per-operator view in Figure 1 is the primary result. 0/600 benign flags gives a 95% upper bound of 0.64%.

Configuration	Harmful flagged (A1–A5, n=4,756)	95% CI	Operators fully caught	Benign flagged (n=600)
C1 hash chain	23.4%	22.2–24.6	6 / 26	0
C2 + replay & rules	34.1%	32.8–35.5	9 / 26	0
C3 shipped, offline	42.6%	41.2–44.0	10 / 26	0
C3h C3 + summary + strict	68.3%	67.0–69.6	17 / 26	0
C4 shipped + anchor (receipt-directed)	70.1%	68.7–71.3	18 / 26	0
C5 C4 + summary + strict	85.5%	84.5–86.5	22 / 26	0
C6 C5 + pinned anchor lookup	**98.1%**	97.7–98.5	25 / 26	0

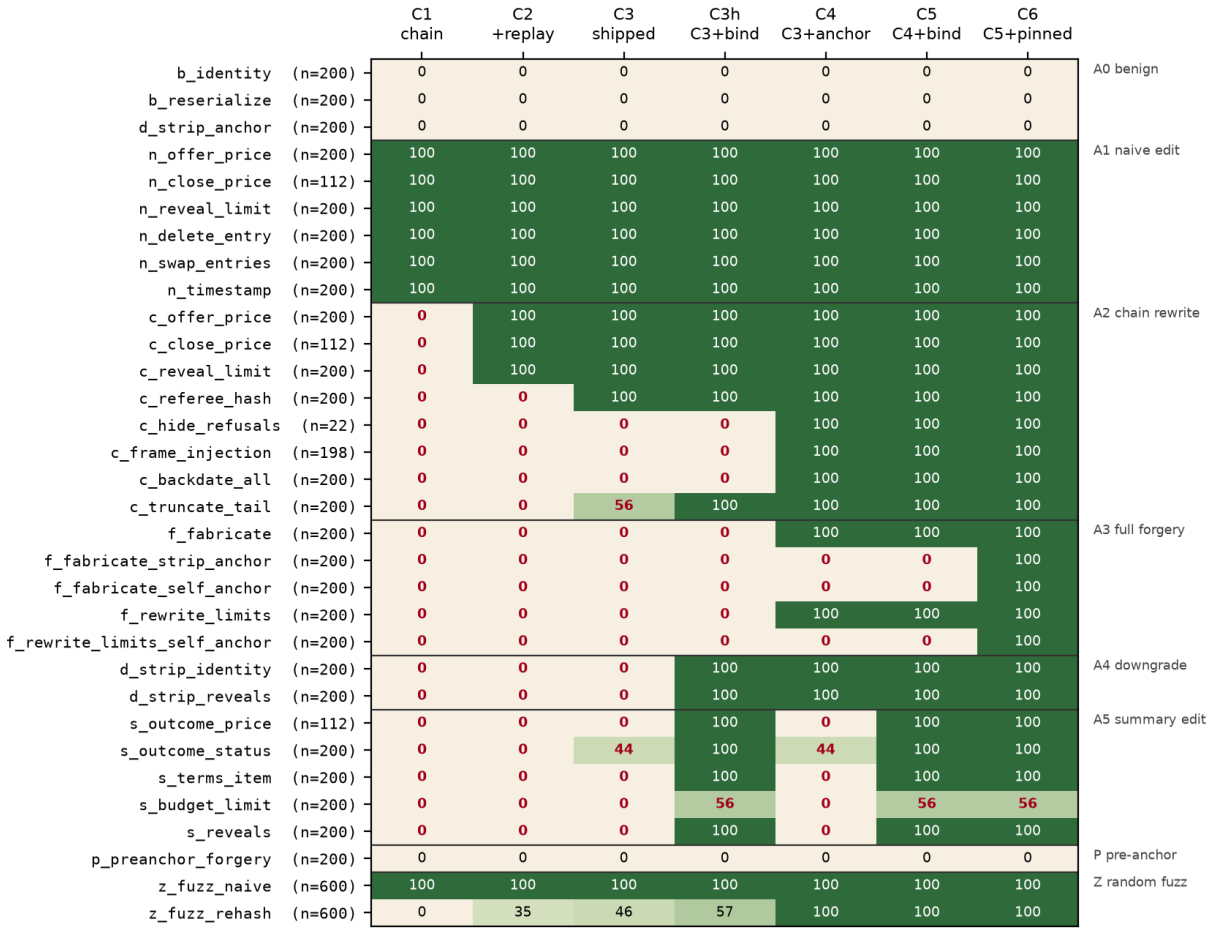


Figure 1. Percentage of mutants flagged, by operator (rows, grouped by adversary class) and verifier configuration (columns; "bind" = summary binding + strict profile, "pinned" = pinned anchor lookup). Red marks a benign record that was flagged, or an in-scope attack that was missed on some mutants. Rows with partial rates (56%, 44%) split exactly by deal outcome: 112 of the 200 deals closed and 88 ended without a deal.

Finding 1: the shipped verifier's verdict is sound against careless editors only. All configurations, including the bare hash chain, catch every naive edit (A1, 1,112/1,112) and flag no benign control. Once

the attacker recomputes hashes (A2), offline detection falls to 61.9% (824/1,332). Replay and signatures catch price and limit edits. Hiding refusal events, inventing an injection attempt blamed on the counterparty, and backdating every timestamp pass all in-record checks, C3h included. Only an anchor catches them.

Finding 2: the receipt directs its own anchor check. With the anchor (C4), every consistent full forgery whose receipt still points at the honest memo is caught (f_fabricate and f_rewrite_limits, 400/400). The same forgeries pass in 100% of cases when the attacker either sets the anchor field to "not anchored" (the check becomes pending) or points it at a memo written from the attacker's own wallet (the shipped check reads the expected writer from the receipt). Under C4 and C5, A3 detection is therefore 40% (400/1,000). Pinning the writer and looking the memo up by deal ID (C6) raises it to 100%, while the benign *anchor-removed* control is still accepted (0/200 flagged), as it should be.

Finding 3: what the receipt card shows is not what is verified. C3 and C4 detect 9.6% of summary edits (88/912), and those only by accident. Flipping a no-deal outcome to "closed" happens to break the round-policy check. A changed price, item, quantity or revealed limit on the card passes. Summary binding catches 90.4% (824/912).

Finding 4: silent downgrades. Deleting the agent identities and offer signatures, or the reveals, turns those checks into "pending" (legacy path). The records then pass C3 in 100% of cases (0/400 flagged). The strict profile catches all 400.

Finding 5: unverifiable claims and the trust boundary. The only in-scope operator that C6 does not fully catch is a changed spending limit on **no-deal** receipts. The card displays that limit, but it is committed and never revealed, so no configuration can check it (C6 catches 112/200, exactly the closed deals). Pre-anchor forgery by the operator itself (P) passes every configuration (0/200). In this corpus agents use per-deal keys, so a forger can mint new ones. Keys bound to registered agent identities would narrow this boundary.

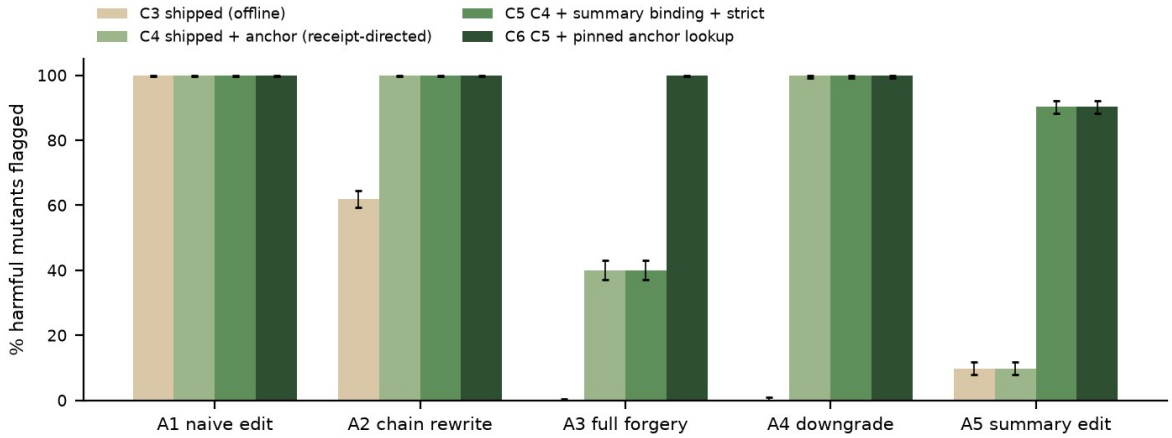


Figure 2. Harmful mutants flagged per adversary class, for the shipped verifier (C3, C4) and the sprint hardenings (C5, C6), with Wilson 95% intervals.

Robustness. On an independent corpus (100 receipts, different seed, 54 closed deals, 3,370 mutants), the pooled rates were 23.4 / 34.1 / 42.5 / 68.2 / 70.1 / 85.4 / 98.1% for C1–C6, and 0/300 benign controls were flagged. Apart from the two random-fuzz operators, every operator-level rate was either 0% or 100%, or equal to the share of closed (or no-deal) deals. The partial rates therefore reflect the structure of the record format, not sampling noise. On the 4 real LLM-agent receipts, the shipped configurations

flagged 0 of 12 benign mutants. The strict profile flagged 9 of 12, because 3 of the 4 real receipts predate agent signatures. This is the main cost of strictness, and it should be shown as "unverifiable (legacy)", not as "tampered".

Live validation on Solana mainnet. On the real deal, the shipped verifier with live reads returned "holds up" for the honest record. It returned "failed" for 8 harmful variants, including the chain-rewrite cases and forgeries that still point at the real memo. It returned "holds up" for 4 harmful variants: a changed outcome price, changed terms, a limits forgery with the anchor field removed, and a limits forgery pointing at a stubbed attacker memo. The pinned lookup read the official wallet's history from mainnet and found exactly one memo for the deal (slot 468,776,619). That memo flagged both forgeries. Summary binding flagged both card edits. The combination flagged all 12 harmful variants and accepted the honest record.

5. Discussion and Limitations

What the badge licenses. The shipped "Record holds up" lets a reader believe that nobody edited the record carelessly after the fact. It does not show that the record is the one that was anchored, that the card matches the log, or that newer evidence was not stripped. These are exactly the properties a counterparty or arbiter would assume. The fix is cheap and does not need more cryptography. The changes are in where the verifier looks for external evidence (a pinned writer and a lookup by ID), in deriving what the user sees from the verified log, and in never showing "pending" with the same green as "verified". As a minimal tamper-evident layer for agent observability formats [7], we suggest four properties: (1) an external anchor that the verifier finds by record ID from a pinned witness; (2) summaries derived from the log, never stored beside it; (3) explicit evidence-version pinning, so evidence cannot be downgraded; and (4) three distinct verdicts: verified, unverifiable and tampered. In-record checks remain useful for a different job. They localise *where* a record breaks and catch referee or agent rule violations before anchoring.

Limitations

The study covers one record format and one verifier, built by the same author. Its generality is argued, not measured. Honest receipts come from scripted agents, not LLMs, since the benchmark targets the verifier and not the negotiators. Only 4 real LLM receipts were tested. The anchor is simulated for the synthetic corpus. Live validation covers one real deal, and its self-anchoring case uses a stubbed attacker transaction. Our operators are hand-designed, and a stronger adversary may find attacks outside them, so detection rates are upper bounds on the soundness of the verdict against attacks we did not model. Pooled rates depend on the operator mix. The hardened checks were designed after seeing the misses, so their 98.1% is in-sample. The second seed and the live run reduce, but do not remove, this optimism. Collusion between the two owners and pre-anchor forgery by the operator are out of scope by construction.

Future Work

We plan to run the benchmark on other agent-record formats, such as OpenTelemetry GenAI traces with a signed-log layer, and to add operators found by an LLM red team. We also plan to bind agent keys to registered identities and to anchor at intervals during a negotiation rather than once at the end, which would shrink the pre-anchor window. Finally, we want to test with users whether a three-state verdict changes what people believe about a deal.

6. Conclusion

We measured what a deployed AI-agent negotiation verifier's green verdict actually guarantees. Against 26 attack operators it is sound only against careless editors: 42.6% of harmful mutants are flagged offline and 70.1% with its anchor. Three small changes, letting the verifier find the anchor for itself, binding what the card shows and refusing silent downgrades, raise this to 98.1% with no false positives on 600 benign controls, and the gaps reproduce on Solana mainnet. Tamper evidence for agent records depends less on more cryptography in the record than on where the verifier looks for evidence and what the verdict shows the user.

Preexisting Work vs. Sprint Work

Table 3. What existed before the sprint and what was done during it. The repository README marks the same split.

Preexisting, before the sprint (SERV Hackathon 02, last commit 8b3bb2a, 13 Nov 2026 04:35 WAT)	**New during the sprint (13–15 Nov 2026, branch apart-epistemics, folder sprint/)**
Blackbox app, LLM agents, referee and schema, hash chain, commit-reveal, agent signatures, spending-limit commitments, replay verifier and CLI (verify.ts), Kite/Solana USDC settlement, Solana memo anchoring, the 4 real receipts, and the existing unit tests, which already included single-edit tamper tests	The research question and adversary taxonomy; the honest-corpus generator; 26 attack operators, the pre-anchor and fuzz operators and the benign controls; the simulated anchor registry; the C1–C6/C3h ablation; the three hardened checks (summary binding, strict profile, pinned anchor lookup); the live mainnet validation; all results, figures and this report. No file in src/ was modified.

Code and Data

- Code repository: https://github.com/Bamioishim/blackbox/tree/apart-epistemics (sprint code in `sprint/`; preexisting code elsewhere)
- Data: `sprint/data/` (honest corpora) and `sprint/results/` (every mutant verdict as CSV, summaries, live-run JSON)
- Live system (preexisting): https://blackbox-serv.vercel.app; real deal verifier: https://blackbox-serv.vercel.app/verify?id=deal_74b9a05fe8f4

Author Contributions

Bamio (solo) designed the study, built the benchmark and wrote the report.

References

[1] Schneier, B. and Kelsey, J. (1999). Secure audit logs to support computer forensics. ACM Transactions on Information and System Security, 2(2), 159–176. https://doi.org/10.1145/317087.317089

[2] Crosby, S. A. and Wallach, D. S. (2009). Efficient Data Structures for Tamper-Evident Logging. 18th USENIX Security Symposium. https://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf

[3] Haber, S. and Stornetta, W. S. (1991). How to time-stamp a digital document. Journal of Cryptology, 3(2), 99–111. https://doi.org/10.1007/BF00196791

[4] Laurie, B., Langley, A. and Kasper, E. (2013). Certificate Transparency. RFC 6962, IETF. https://www.rfc-editor.org/rfc/rfc6962

[5] Newman, Z., Meyers, J. S. and Torres-Arias, S. (2022). Sigstore: Software Signing for Everybody. ACM CCS 2022. https://doi.org/10.1145/3548606.3560596

[6] Coalition for Content Provenance and Authenticity (2025). C2PA Technical Specification, v2.x. https://c2pa.org/specifications/

[7] OpenTelemetry (2025). Semantic conventions for generative AI systems. https://opentelemetry.io/docs/specs/semconv/gen-ai/

[8] Hammond, L. et al. (2025). Multi-Agent Risks from Advanced AI. Cooperative AI Foundation Technical Report #1. arXiv:2502.14143. https://arxiv.org/abs/2502.14143

[9] Cotton-Barratt, O. and Finnveden, L. (2026). AI for AI for Epistemics. Forethought. https://www.forethought.org/research/ai-for-ai-for-epistemics

[10] Greshake, K. et al. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISec '23. arXiv:2302.12173. https://arxiv.org/abs/2302.12173

[11] DeMillo, R. A., Lipton, R. J. and Sayward, F. G. (1978). Hints on test data selection: Help for the practicing programmer. IEEE Computer, 11(4), 34–41. https://doi.org/10.1109/C-M.1978.218136

[12] Jia, Y. and Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. IEEE Transactions on Software Engineering, 37(5), 649–678. https://doi.org/10.1109/TSE.2010.62

[13] Bhargavan, K., Brzuska, C., Fournet, C., Green, M., Kohlweiss, M. and Zanella-Béguelin, S. (2016). Downgrade Resilience in Key-Exchange Protocols. IEEE Symposium on Security and Privacy. https://doi.org/10.1109/SP.2016.37

Appendix A. Limitations and Dual-Use / Ethical Considerations

What the result does and does not establish. It establishes, for one deployed verifier and record format, which hand-designed adversary classes its "holds up" verdict excludes, and how far three specific changes improve that. It does not establish security against adversaries outside our operators. It does not establish that the hardened verifier is complete, since the hardenings were designed after seeing the misses. It also does not show that the results generalise to other agent-record systems, or that records reflect what agents and owners intended. Collusion between owners and forgery by the anchoring operator before anchoring are not detectable by any configuration tested.

Dual use. The operators are, by design, a catalogue of ways to forge agent-negotiation records that a verifier will accept. Someone could use them against Blackbox or similar systems. We judge the risk low and the defensive value high, for four reasons. The attacks are simple once the record format is public. The gaps are in the author's own system, which handles at most 1 USDC per real deal. The fixes are described next to the attacks. And publishing verdict-soundness results is how users learn what a badge does not guarantee. We did not attack any third-party system. We wrote nothing to any blockchain during the sprint: the self-anchoring attack used a locally stubbed transaction, and all mainnet access was read-only. Neither the anchor wallet nor any funds were used. The hardened checks have not yet been

deployed to production. Until they are, the live verifier has the gaps described here, and this report says so plainly.

Manipulation risk. A verifier badge can manipulate people as well as inform them, because a green "holds up" invites more trust than the checks warrant. Our main recommendation is a three-state verdict (verified, unverifiable, tampered). It is meant to reduce that risk.

Personal data. No human-subject data was collected. Synthetic receipts contain only generated values. The real receipts are the author's own deals and contain public on-chain addresses, shortened account IDs and no email addresses. No private keys or secrets are included in the code, data or report.

Appendix B. Reproduction

From the repository root on branch `apart-epistemics`: `npm ci`, then `npx tsx --conditions=react-server sprint/bench/corpus.mts 200 20261113`, then `npx tsx --conditions=react-server sprint/bench/run.mts sprint/data/corpus.json 7`, then `python3 sprint/analysis/analyze.py`, then (optionally, read-only RPC) `npx tsx --conditions=react-server sprint/bench/live-real.mts`. The full benchmark took about 47 s on the author's machine. Keys and salts are random on each run, so hashes differ between runs, but the operator-level rates do not. Every per-mutant verdict, with the checks that fired, is in `sprint/results/rows_synthetic.csv`.

LLM Usage Statement

An AI assistant (Grok Bot) helped write the benchmark code, run the experiments and draft this report, under the author's direction. All numbers in the report come from the committed result files, and every reference was checked to exist. The author is responsible for all claims.